



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Кулахсзян Сергей Грайрович

ЭМПИРИЧЕСКАЯ ФУНКЦИЯ РАСПРЕДЕЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 3

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

г. Владивосток
2026

Содержание

1	Построение эмпирической функции распределения с доверительным интервалом	2
2	Сравнение истинной функции распределения и эмпирической функций распределения	3
3	Сравнение истинной функции распределения и эмпирической функции распределения из <code>scipy</code>	6

1 Построение эмпирической функции распределения с доверительным интервалом

Эмпирической функцией распределения является функция вида:

$$F^*(x) = \frac{m_x}{n},$$

где m_x – число вариантов выборки меньше или равно x , n – объём выборки.

Доверительным интервалом функции распределения является вероятность неравенства:

$$P(U(x) < F^*(x) < L(x)) \geq 1 - \alpha,$$

где $1 - \alpha$ – уровень доверия, $(U(x), L(x))$ – доверительный интервал, вычисляемый по формуле:

$$L(x) = \max(F^*(x) - \varepsilon; 0)$$

$$U(x) = \min(F^*(x) + \varepsilon; 1)$$

$$\varepsilon = \sqrt{\frac{1}{2n} \ln \left(\frac{2}{\alpha} \right)}$$

Таким образом код для него вышел таким:

```
1 def empiric_distribution_function(data: np.ndarray, x: float,
2   alpha: float = 0.05) -> tuple[int, tuple[float, float]]:
3     data = np.sort(data)
4     n = len(data)
5
6     value = np.where(data <= x)[0].size / n
7
8     epsilon = np.sqrt(np.log(2 / alpha) / (2 * n))
9
10    lower_bound = max(0, value - epsilon)
11    upper_bound = min(1, value + epsilon)
12
13    return value, (lower_bound, upper_bound)
```

2 Сравнение истинной функции распределения и эмпирической функций распределения

Код:

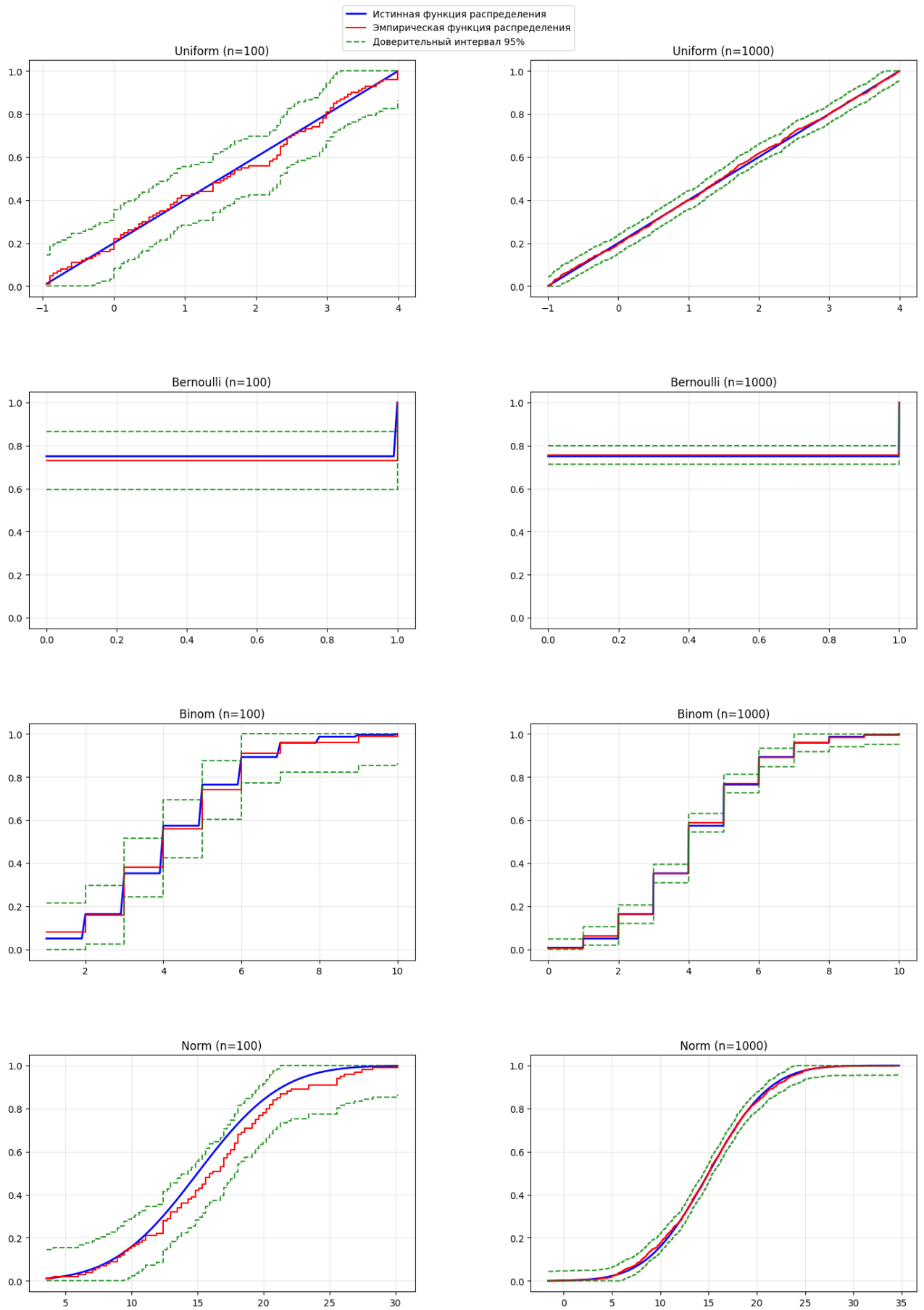
```
1 def plot_all_distribution_functions(data: dict[str, dict[int,
2     dict]]) -> None:
3     fig, axes = plt.subplots(4, 2, figsize=(17, 22))
4     fig.subplots_adjust(hspace=0.4, wspace=0.3)
5
6     distribution_names = ["uniform", "bernoulli", "binom",
7         "norm"]
8     sizes = [100, 1000]
9
10    for i, distribution in enumerate(distribution_names):
11        for j, size in enumerate(sizes):
12            ax = axes[i, j]
13            sample = data[distribution][size]["X"]
14            sample_sorted = np.sort(sample)
15
16            x_min, x_max = sample_sorted[0], sample_sorted[-1]
17            x_range = np.linspace(x_min, x_max, size)
18
19            if distribution == "uniform":
20                true_cdf = stats.uniform.cdf(x_range, loc=-1,
21                    scale=5)
22            elif distribution == "bernoulli":
23                true_cdf = stats.bernoulli.cdf(x_range,
24                    p=0.25)
25            elif distribution == "binom":
26                true_cdf = stats.binom.cdf(x_range, p=0.25,
27                    n=17)
28            elif distribution == "norm":
29                true_cdf = stats.norm.cdf(x_range, loc=15,
30                    scale=5)
31
32            l1, = ax.plot(x_range, true_cdf, color='blue',
33                label='Истинная функция распределения',
34                linewidth=2)
```

```

27
28     values = []
29     lower_bounds = []
30     upper_bounds = []
31
32     for x in x_range:
33         val, (low, upp) =
34             empiric_distribution_function(sample, x)
35         values.append(val)
36         lower_bounds.append(low)
37         upper_bounds.append(upp)
38
39     l2, = ax.step(x_range, values, color='red',
40                  label='Эмпирическая функция распределения',
41                  where='post')
42
43     l3, = ax.step(x_range, lower_bounds, color='green',
44                  linestyle='--',
45                  label='Доверительный интервал 95%', alpha=0.8,
46                  where='post')
47
48     ax.step(x_range, upper_bounds, color='green',
49            linestyle='--', alpha=0.8, where='post')
50
51     ax.set_title(f"{distribution.capitalize()} (n={size})")
52     ax.set_ylim(-0.05, 1.05)
53     ax.grid(True, alpha=0.3)
54     if i == 0 and j == 0:
55         lines = [l1, l2, l3]
56         labels = [l.get_label() for l in lines]
57
58     fig.subplots_adjust(hspace=0.4, wspace=0.3, top=0.94)
59     fig.legend(lines, labels, loc='upper center', ncol=1,
60              frameon=True, bbox_to_anchor=(0.5, 0.98))
61     plt.show()
62
63 plot_all_distribution_functions(data)

```

Результат графиков:



3 Сравнение истинной функции распределения и эмпирической функции распределения из scipy

Код:

```
1 def plot_all_distribution_functions_scipy(data: dict[str,
2 dict[int, dict]]) -> None:
3     fig, axes = plt.subplots(4, 2, figsize=(17, 22))
4     fig.subplots_adjust(hspace=0.4, wspace=0.3)
5
6     distribution_names = ["uniform", "bernoulli", "binom",
7 "norm"]
8     sizes = [100, 1000]
9
10    for i, distribution in enumerate(distribution_names):
11        for j, size in enumerate(sizes):
12            ax = axes[i, j]
13            sample = data[distribution][size]["X"]
14            sample_sorted = np.sort(sample)
15
16            x_min, x_max = sample_sorted[0], sample_sorted[-1]
17            x_range = np.linspace(x_min, x_max, size)
18
19            if distribution == "uniform":
20                true_cdf = stats.uniform.cdf(x_range, loc=-1,
21 scale=5)
22            elif distribution == "bernoulli":
23                true_cdf = stats.bernoulli.cdf(x_range,
24 p=0.25)
25            elif distribution == "binom":
26                true_cdf = stats.binom.cdf(x_range, p=0.25,
27 n=17)
28            elif distribution == "norm":
29                true_cdf = stats.norm.cdf(x_range, loc=15,
30 scale=5)
31
32            l1, = ax.plot(x_range, true_cdf, color='blue',
33 label='Истинная функция распределения',
34 linewidth=2)
```

```

27
28         ecdf_result = stats.ecdf(sample)
29         values = ecdf_result.cdf.evaluate(x_range)
30         continued_interval =
31         ecdf_result.cdf.confidence_interval(confidence_level=0.9
32
33         l2, = ax.step(x_range, values, color='red',
34         label='Эмпирическая функция распределения scipy.stats',
35         where='post')
36
37         l3, = ax.step(x_range,
38         continued_interval.low.evaluate(x_range),
39         color='green', linestyle='--',
40         label='Доверительный интервал 95% scipy.stats',
41         alpha=0.8, where='post')
42         ax.step(x_range,
43         continued_interval.high.evaluate(x_range),
44         color='green', linestyle='--', alpha=0.8,
45         where='post')
46
47         ax.set_title(f"{distribution.capitalize()} (n={size})")
48         ax.set_ylim(-0.05, 1.05)
49         ax.grid(True, alpha=0.3)
50         if i == 0 and j == 0:
51             lines = [l1, l2, l3]
52             labels = [l.get_label() for l in lines]
53
54     fig.subplots_adjust(hspace=0.4, wspace=0.3, top=0.94)
55     fig.legend(lines, labels, loc='upper center', ncol=1,
56     frameon=True, bbox_to_anchor=(0.5, 0.98))
57     plt.show()
58
59 plot_all_distribution_functions_scipy(data)

```

Результат графиков:

